

Objetivo do Teste

Mais do que apenas entregar uma solução que "funcione", este teste tem como objetivo avaliar como você pensa, como estrutura seu código e como toma decisões técnicas em toda a stack.

Queremos entender sua capacidade de projetar e implementar uma solução completa de ponta a ponta: uma API robusta no backend e uma interface dinâmica e responsiva no frontend, com integração fluida entre elas.

Lembre-se: em um cenário real, seu código será lido, mantido e evoluído por outras pessoas do time, tanto no front quanto no back.

O Problema

Você está desenvolvendo o motor principal de um serviço financeiro responsável por gerenciar e atualizar saldos de contas bancárias. Sua tarefa é construir uma solução completa: uma API REST que receba requisições de transações financeiras e uma aplicação web em Angular que consuma essa API e permita ao usuário visualizar e movimentar as contas.

O Evento de Entrada (Backend)

Sua API deve expor um endpoint que recebe o seguinte payload em formato JSON:

```
JSON
{
  "eventId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "accountId": "7b895f64-5717-4562-b3fc-2c963f66afa7",
  "type": "CREDIT | DEBIT",
  "amount": 150.75,
  "occurredAt": "2026-01-30T10:15:00Z"
}
```

A Aplicação Web (Frontend)

A interface deve permitir, no mínimo:

- **Listagem de contas:** exibir as contas cadastradas com seus saldos atuais consolidados.
- **Extrato da conta:** exibir o histórico de transações de uma conta (com paginação), evidenciando o impacto de cada lançamento no saldo.



- **Lançamento de transações:** formulário para envio de novos eventos (crédito ou débito), com validação de campos e feedback claro de sucesso, processamento ou erro — incluindo eventos duplicados e saldo insuficiente.
- **Estados de interface:** a aplicação deve tratar adequadamente carregamento, erros de comunicação e respostas de validação da API.

Regras de Negócio Obrigatórias

A sua solução funcional deve garantir quatro pilares essenciais:

1. **Idempotência:** um evento (identificado unicamente pelo eventId) nunca pode ser processado mais de uma vez. O sistema deve estar protegido contra chamadas duplicadas.
2. **Consistência:** o saldo da conta não pode ficar negativo e deve refletir exatamente o histórico de transações.
3. **Transacionalidade:** a gravação do histórico da transação e a atualização do saldo consolidado devem ocorrer de forma transacional no banco de dados. Ou tudo funciona, ou nada é gravado.
4. **Integridade ponta a ponta:** a interface deve refletir fielmente o estado real do backend. As validações do frontend melhoram a experiência, mas o backend é a fonte da verdade — inclusive para rejeitar eventos duplicados ou débitos sem saldo suficiente.

Requisitos Técnicos

A solução deve conter as seguintes tecnologias e práticas:

Backend

- **Linguagem e Framework:** API RESTful desenvolvida em C# utilizando o ecossistema .NET (ASP.NET Core).
- **Documentação:** API documentada utilizando OpenAPI/Swagger.
- **Persistência:** Banco de dados PostgreSQL, manipulado via Entity Framework Core (com migrations e mapeamento adequados).
- **Testes:** Testes unitários utilizando xUnit, Moq, FluentAssertions ou equivalentes.

Frontend

- **Framework:** Angular (versão recente) com TypeScript, utilizando componentes, serviços e roteamento.
- **Reatividade:** uso adequado de RxJS (Observables) no consumo da API.
- **UI:** interface responsiva com HTML5 e CSS3/SASS, utilizando Angular Material (ou biblioteca de componentes equivalente).
- **Testes:** testes unitários e de componentes com Jasmine/Karma ou Jest.



Ambiente

- A aplicação (frontend e backend) e o banco de dados devem subir facilmente utilizando Docker e Docker Compose.

O Que Vamos Avaliar (Critérios de Sucesso)

Não iremos olhar apenas para o resultado final. Nossa avaliação técnica irá focar em:

- **Design de Código e Arquitetura:** Como você separa responsabilidades nos dois lados da stack (inversão e injeção de dependência no backend; componentização, serviços e modelos tipados no frontend) e se aplica conceitos de arquitetura limpa ou DDD para organizar o domínio financeiro.
- **Boas Práticas e Clean Code:** Legibilidade, nomenclatura de variáveis e métodos, tratamento de erros adequado e aderência a padrões de estilo — tanto em C# quanto em TypeScript.
- **Modelagem de Dados:** Como você estruturou as tabelas no PostgreSQL (índices, relações) e como configurou o EF Core.
- **Integração e Experiência de Uso:** Como o frontend consome a API (contrato, tratamento de erros, estados de carregamento) e a fluidez da experiência do usuário na interface.
- **Qualidade dos Testes:** Mais do que cobertura percentual, queremos ver se os testes validam os cenários críticos — no backend (saldo insuficiente, tentativa de duplicidade, concorrência básica) e no frontend (validações do formulário, renderização dos estados e comportamento dos serviços).
- **Comunicação (O README):** Sua capacidade de explicar as decisões tomadas, os trade-offs escolhidos e como rodar o projeto.

Diferenciais Opcionais

Estes itens não são obrigatórios para a aprovação, mas demonstrarão um nível de maturidade técnica que valorizamos muito:

- **Gerência de estado:** utilização de NgRx (ou equivalente) para um gerenciamento de estado previsível no Angular.
- **Autenticação e autorização:** proteção da aplicação (front e back) com Keycloak, utilizando OIDC/OAuth2.
- **Mensageria:** Receber o payload na API e enviá-lo para uma fila do RabbitMQ para processamento assíncrono do saldo.
- **Cache:** Implementação de caching ou rate limiting utilizando Redis.
- **Observabilidade:** Logs estruturados indexados no Elasticsearch e implementação de Health Checks.



Entrega

- Suba o código em um repositório público no GitHub e nos envie o link.
- Inclua um arquivo README.md detalhado explicando sua arquitetura de ponta a ponta, as decisões técnicas e os trade-offs, além do passo a passo (via docker compose up) para rodar a aplicação completa e os testes (backend e frontend).
- Prazo máximo para entrega: Uma semana (7 dias) após o recebimento do teste.